

Analyser — API Reference

Version: 1.0 · **Last updated:** May 2026

The Analyser API provides server-side endpoints for the cross-device label sync feature (PR #80). All routes are SvelteKit server endpoints located under `src/routes/api/`.

Table of Contents

- 1. Authentication
- 2. Endpoints
 - GET `/api/labels/[uuid]`
 - PUT `/api/labels/[uuid]`
 - GET `/api/labels/resolve/[code]`
- 3. Error Responses
- 4. Rate Limiting
- 5. Data Expiry

1. Authentication

These endpoints use no authentication tokens or API keys. Access control is by obscurity of the UUID — only a client that knows the UUID can read or write its labels.

UUIDs are randomly generated (`crypto.randomUUID()`) in the browser and stored in `localStorage`. They are never transmitted except in the URL path of these API calls and in the `?sync=` query parameter used for device-linking.

2. Endpoints

GET `/api/labels/[uuid]`

Returns the stored label map for a sync identity.

Path parameter

Parameter	Format	Description
<code>uuid</code>	UUIDv4 (<code>xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx</code>)	The sync identity UUID

Responses

200 OK

```
{
  "labels": {
    "garmin:3151:120": "Polar H10",
    "garmin:3151:11": "Assioma Duo"
  }
}
```

The `labels` object is a `Record<string, string>` mapping device keys (as produced by `deviceKey()` in `src/lib/utils/deviceChannels.ts`) to user-assigned label strings.

400 Bad Request — UUID does not match the expected format.

```
{ "error": "Invalid UUID format" }
```

404 Not Found — No labels stored for this UUID (new identity, or TTL expired).

```
{ "error": "Not found" }
```

500 Internal Server Error — Redis credentials not configured or database unreachable.

```
{ "error": "Internal server error" }
```

PUT /api/labels/[uuid]

Stores or overwrites the label map for a sync identity. Also refreshes the short-code reverse-lookup index.

This endpoint is called by `pushLabels()` in `src/lib/stores/sync.ts` — once on first visit to seed the remote, then automatically after every device label change.

Path parameter

Parameter	Format	Description
<code>uuid</code>	UUIDv4	The sync identity UUID

Request body (`Content-Type: application/json`)

```
{
  "labels": {
    "garmin:3151:120": "Polar H10"
  },
  "shortCode": "E6Y-NXEMF"
}
```

Field	Type	Validation
labels	Record<string, string>	All values must be strings
shortCode	string	Must match XXX-XXXXX (3 alphanumeric, dash, 5 alphanumeric)

Responses

200 OK

```
{ "ok": true }
```

400 Bad Request — Invalid UUID, malformed JSON, or body fails validation.

```
{ "error": "Body must contain a labels object (string → string) and a shortCode string" }
```

500 Internal Server Error

Side effects

Two Redis keys are written (or refreshed) atomically:

- labels:{uuid} — the label map, with a rolling 90-day TTL
- code:{shortCode} — the UUID string, with a rolling 90-day TTL

GET /api/labels/resolve/[code]

Resolves a short sync code to its full UUID. Used when a user types a short code in the Sync panel's "Link another device" input.

Path parameter

Parameter	Format	Description
code	XXX-XXXXX (e.g. E6Y-NXEMF)	The short sync code

Responses

200 OK

```
{ "uuid": "a1b2c3d4-e5f6-7890-abcd-ef1234567890" }
```

400 Bad Request — Code does not match the expected format.

```
{ "error": "Invalid sync code format – expected XXX-XXXXX (8 alphanumeric chars with dash)" }
```

404 Not Found — Code not found in Redis (never stored, or 90-day TTL expired).

```
{ "error": "Sync code not found or expired" }
```

429 Too Many Requests — Rate limit exceeded (10 requests per minute per IP).

```
{ "error": "Too many requests – please wait before trying again" }
```

Response includes `Retry-After: 60` header.

500 Internal Server Error

3. Error Responses

All error responses follow the same shape:

```
{ "error": "Human-readable description" }
```

HTTP status codes used:

Code	Meaning
400	Invalid input (bad UUID format, malformed body, invalid short code)
404	Resource not found (no data for UUID, unknown short code)
429	Rate limit exceeded
500	Server error (Redis unavailable or misconfigured)

4. Rate Limiting

The resolve endpoint (`GET /api/labels/resolve/[code]`) is rate-limited to **10 requests per minute per IP address** to deter brute-force code enumeration.

The rate limiter is **Redis-backed** (Upstash) and enforced globally across all Vercel serverless instances. It uses an atomic `INCR` + `EXPIRE` pipeline per IP key (`ratelimit:resolve:{ip}`), so the limit cannot be bypassed by routing requests to different function instances. If Redis is unavailable, the limiter **fails open** — requests are allowed through rather than blocked, preserving availability.

The `Retry-After` header value is derived from the Redis key's remaining TTL at the time of the 429 response.

The `GET /api/labels/[uuid]` and `PUT /api/labels/[uuid]` endpoints are not rate-limited. The UUID namespace ($36^8 \times 36^8$ entries) makes enumeration infeasible.

5. Data Expiry

All Redis entries use a **90-day rolling TTL**. The TTL is refreshed on every successful PUT, so active users never lose their labels. Inactive identities expire automatically with no explicit cleanup required.

After expiry:

- `GET /api/labels/{uuid}` returns 404 — treated by the client as "no remote data yet"
- `GET /api/labels/resolve/{code}` returns 404 — user sees "Code not found or expired"